ĐẠI HỌC BÁCH KHOA HÀ NỘI

BÁO CÁO PROJECT 1

Phân loại chữ số viết tay trong dataset MNIST bằng phương pháp SVM và CNN

PHẠM HOÀNG THÁI DƯƠNG
duong.pht235061@sis.hust.edu.vn

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: CN. Trần Thị Lan Anh

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 01/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

BÁO CÁO PROJECT 1

Phân loại chữ số viết tay trong dataset MNIST bằng phương pháp SVM và CNN

PHẠM HOÀNG THÁI DƯƠNG
duong.pht235061@sis.hust.edu.vn

Khoa học máy tính

Giảng viên hướng dẫn: CN. Trần Thị Lan Anh _____

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 01/2026

MỤC LỤC

Chương 1: Giới thiệu đề tài 1
1.1: Đặt vấn đề 1
1.2: Mục tiêu và định hướng giải pháp 1
1.3: Bố cục đồ án 1

Chương 2: Cơ sở lý thuyết 2
2.1: Support Vector Machine (SVM) 2
2.2: Convolutional Neural Network (CNN) 5

Chương 3: Áp dụng vào bài toán thực tế 8
3.1: Thực nghiệm với mô hình SVM 8
3.2: Thực nghiệm với mô hình CNN 10

Chương 4: Kết quả thực nghiệm 13
4.1: Kết quả thực nghiệm với mô hình SVM 13
4.2: Kết quả thực nghiệm với mô hình CNN 14

Chương 5: Demo 16
5.1: Huấn luyện mô hình 16
5.2: Kiểm tra mô hình 16

Chương 6: Kết luận 17

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Nhận dạng chữ số viết tay là bước đầu tiên của các hệ thống OCR (Optical character recognition – nhận diện ký tự quang học) trong ngân hàng, bưu điện. Tuy nhiên, sự khác biệt trong nét vẽ, độ nghiêng của nét chữ số, hay độ nhiễu trong hình ảnh khiến việc nhận diện không đơn giản.

1.2 Mục tiêu và định hướng giải pháp

Để giải quyết vấn đề này, ta đến với bài toán phân loại đa lớp. Trong học máy, đây là bài toán phân loại các mẫu dữ liệu vào một trong nhiều hơn hai lớp đối tượng, với nhiều hướng giải quyết, bao gồm các phương pháp thiên về học máy truyền thống, và các phương pháp thiên về học sâu. Với mục đích hiểu sâu hơn về cả học máy truyền thống và học sâu, em đã lựa chọn giải quyết bài toán bằng hai phương pháp: Support vector machine (SVM) và Convolutional neural network (CNN), lần lượt là các phương pháp được ứng dụng phổ biến trong học máy truyền thống và học sâu. Về bộ dữ liệu dành cho huấn luyện và kiểm tra, dataset MNIST, với 70000 ảnh chữ số viết tay đen trắng kích thước 24x24 pixel, là một lựa chọn phù hợp cho bài toán.

1.3 Bố cục đồ án

Báo cáo được chia thành 6 chương như sau:

Chương 2 trình bày cơ sở lý thuyết của hai phương pháp chính được sử dụng: Support Vector Machine (SVM) và Convolutional Neural Network (CNN), bao gồm các khái niệm, công thức toán học và nguyên lý hoạt động.

Chương 3 mô tả quá trình áp dụng hai mô hình vào bài toán phân loại chữ số MNIST, từ tiền xử lý dữ liệu, thiết kế kiến trúc mạng, đến các tham số huấn luyện và triển khai code.

Chương 4 trình bày kết quả thực nghiệm, bao gồm độ chính xác, thời gian huấn luyện, ma trận nhầm lẫn và phân tích sai số của cả hai mô hình, đồng thời so sánh hiệu năng giữa SVM và CNN.

Chương 5 minh họa giao diện demo để kiểm tra mô hình trên các mẫu dữ liệu thực tế, cho thấy khả năng ứng dụng của hệ thống.

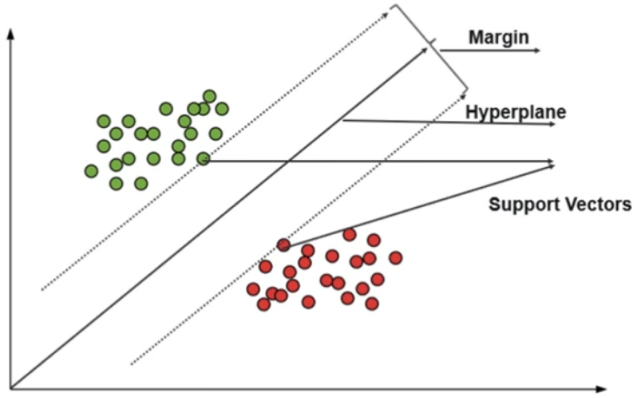
Chương 6 tổng kết các kết quả đạt được.

CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT

2.1 Support Vector Machine (SVM)

2.1.1 Bài toán phân loại tuyến tính

Giả sử chúng ta có hai lớp (class) khác nhau được mô tả bởi các điểm trong không gian nhiều chiều. Trong trường hợp hai lớp này là phân tách tuyến tính (*linearly separable*), tồn tại một siêu phẳng (hyperplane) có thể phân chia chính xác các điểm dữ liệu thuộc về hai lớp đó.



Hình 2.1: Minh họa siêu phẳng phân chia trong không gian hai chiều.

Trên thực tế, có thể tồn tại nhiều mặt phân chia khác nhau. Thuật toán SVM tìm kiếm một siêu phẳng tối ưu sao cho khoảng cách từ điểm gần nhất của mỗi lớp đến mặt phân chia là bằng nhau và khoảng cách đó (gọi là lề - *margin*) đạt giá trị lớn nhất có thể.

2.1.2 Xây dựng bài toán tối ưu

Giả sử tập dữ liệu huấn luyện bao gồm các cặp dữ liệu $(\mathbf{x}_n, y_n)$ với $n = 1, 2, \ldots, N$, trong đó $\mathbf{x}_n \in \mathbb{R}^d$ là vector đầu vào d chiều và $y_n \in \{1, -1\}$ là nhãn.

Mục tiêu của thuật toán SVM là tìm bộ tham số $(\mathbf{w}, b)$ sao cho khoảng cách từ các điểm gần siêu phẳng nhất đến siêu phẳng đó là lớn nhất. Giả thuyết về lề hàm số cho phép chuẩn hóa sao cho $y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 1$ cho mọi điểm, dẫn đến bài toán tối ưu chuẩn tắc:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|_2^2 \quad \text{thỏa mãn } 1 - y_n\left(\sum_{j=1}^{d} w^{(j)}x_n^{(j)} + b\right) \leq 0, \forall n \tag{2.1}$$

2.1.3 Bài toán đối ngẫu

Để giải bài toán tối ưu lồi với các ràng buộc bất đẳng thức, chúng ta sử dụng phương pháp nhân tử Lagrange để chuyển sang bài toán đối ngẫu. Điều kiện Slater được thỏa mãn, đảm bảo tính chất đối ngẫu mạnh.

Bài toán đối ngẫu cuối cùng được phát biểu dưới dạng tối ưu hóa hàm mục tiêu $g(\boldsymbol{\lambda})$ như sau:

$$\boldsymbol{\lambda}^* = \arg\max_{\boldsymbol{\lambda}} \left(\sum_{n=1}^{N} \lambda_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} \lambda_n \lambda_m y_n y_m \mathbf{x}_n^T \mathbf{x}_m \right) \tag{2.2}$$

Thỏa mãn các điều kiện ràng buộc sau:

- $0 \leq \lambda_n \leq C, \quad \forall n = 1, 2, \ldots, N$
- $\sum_{n=1}^{N} \lambda_n y_n = 0$

Vector hỗ trợ và Xác định tham số mô hình

Những điểm dữ liệu ứng với $\lambda_n > 0$ được gọi là các **Vector hỗ trợ (Support Vectors)**. Đây là những điểm nằm gần siêu phẳng phân chia nhất và trực tiếp quyết định vị trí của siêu phẳng đó.

Sau khi giải được $\boldsymbol{\lambda}$ bằng thư viện tối ưu, ta xác định các tham số của mô hình như sau:

1. **Tìm $\mathbf{w}$:** Thay các giá trị λ_n vừa tìm được vào công thức đạo hàm:

$$\mathbf{w}^* = \sum_{n \in \mathcal{S}} \lambda_n y_n \mathbf{x}_n \tag{2.3}$$

   với $\mathcal{S}$ là tập chỉ số của các Support Vectors.

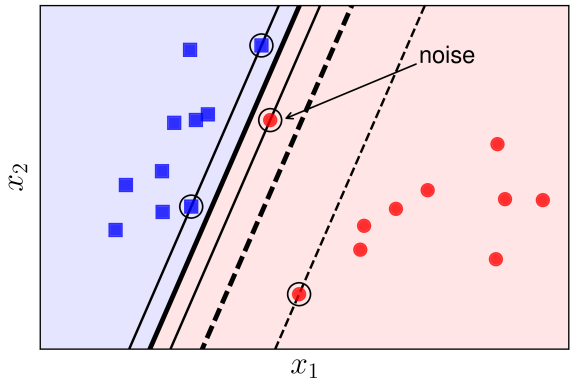
2. **Tìm b:** Với bất kỳ một Support Vector $\mathbf{x}_k$ ($k \in \mathcal{S}$), ta có phương trình $y_k(\mathbf{w}^T \mathbf{x}_k + b) = 1$. Nhân cả hai vế với y_k (do $y_k^2 = 1$), ta thu được:

$$b^* = y_k - \mathbf{w}^{*T} \mathbf{x}_k = y_k - \sum_{j=1}^{d} w^{*(j)} x_k^{(j)} \tag{2.4}$$

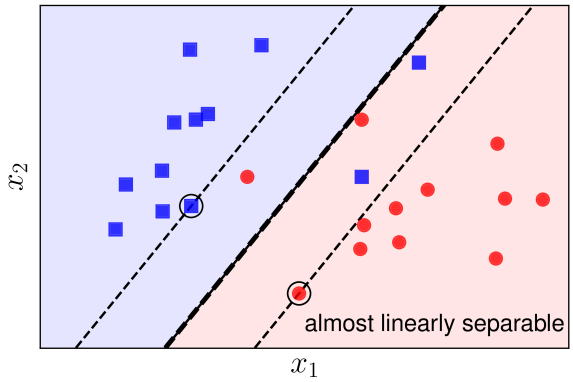
   Để kết quả ổn định hơn về mặt số học, giá trị b^* thường được tính bằng cách lấy trung bình cộng trên toàn bộ tập Support Vectors $\mathcal{S}$.

2.1.4 Soft Margin Support Vector Machine

Trong thực tế, phương pháp Hard Margin SVM gặp phải hai hạn chế lớn: dữ liệu phân tách tuyến tính nhưng có nhiễu, hoặc dữ liệu không phân tách tuyến tính.



Hình 2.2: Ảnh hưởng của điểm nhiễu đối với Hard Margin SVM.



Hình 2.3: Dữ liệu không thể phân tách tuyến tính.

Để giải quyết, khái niệm Soft Margin được đưa ra. Với mỗi điểm dữ liệu $\mathbf{x}_n$, chúng ta giới thiệu biến bù $\xi_n \geq 0$ đại diện cho độ vi phạm. Ràng buộc mềm là $y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 1 - \xi_n$.

Tham số $C > 0$ cân bằng giữa tối đa hóa lề và cực tiểu hóa sai số. Bài toán tối ưu là:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{n=1}^{N} \xi_n \tag{2.5}$$

Bài toán đối ngẫu cuối cùng:

$$\lambda = \arg\max_{\lambda} \left(\sum_{n=1}^{N} \lambda_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} \lambda_n \lambda_m y_n y_m x_n^T x_m \right) \tag{2.6}$$

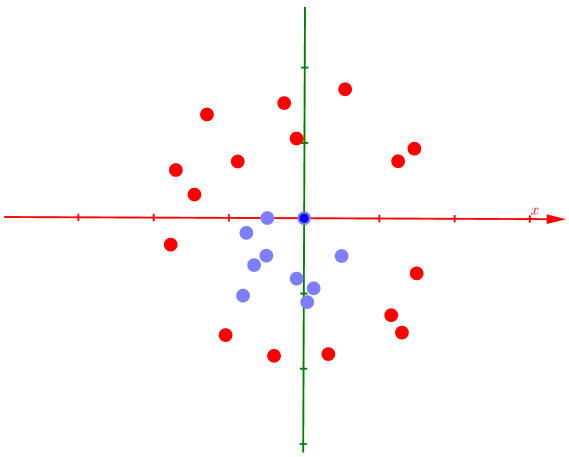
với $0 \leq \lambda_n \leq C$ và $\sum \lambda_n y_n = 0$.

Phân tích vị trí các điểm dữ liệu dựa trên KKT:

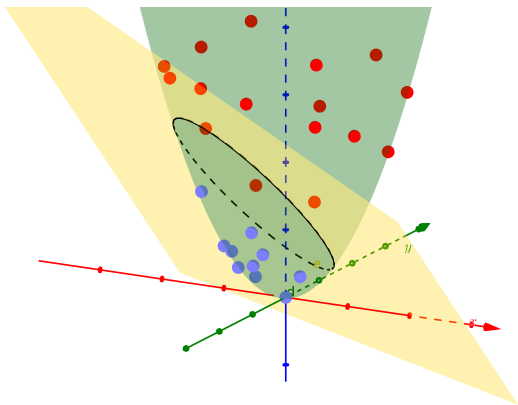
- $\lambda_n = 0$: Điểm nằm trong vùng an toàn (đúng phía và ngoài lề).
- $0 < \lambda_n < C$: Điểm nằm chính xác trên đường margin.
- $\lambda_n = C$: Điểm vi phạm margin (nhiễu hoặc phân lớp sai).

2.1.5 Kernel Support Vector Machine

Kernel SVM giải quyết vấn đề khi dữ liệu không phân tách tuyến tính bằng cách ánh xạ sang không gian cao chiều.



Hình 2.4: Dữ liệu phi tuyến tính trong không gian 2 chiều.



Hình 2.5: Dữ liệu được ánh xạ sang không gian 3 chiều

Ý tưởng là ánh xạ dữ liệu qua hàm $\Phi(\mathbf{x})$ để phân tách tuyến tính trong không gian mới.

Hàm Kernel $K(\mathbf{x}, \mathbf{z}) = \Phi(\mathbf{x})^T \Phi(\mathbf{z})$ cho phép tính tích vô hướng mà không cần ánh xạ trực tiếp.

Bài toán tối ưu với Kernel:

$$\lambda = \arg\max_{\lambda} \left(\sum_{n=1}^{N} \lambda_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} \lambda_n \lambda_m y_n y_m K(\mathbf{x}_n, \mathbf{x}_m) \right) \tag{2.7}$$

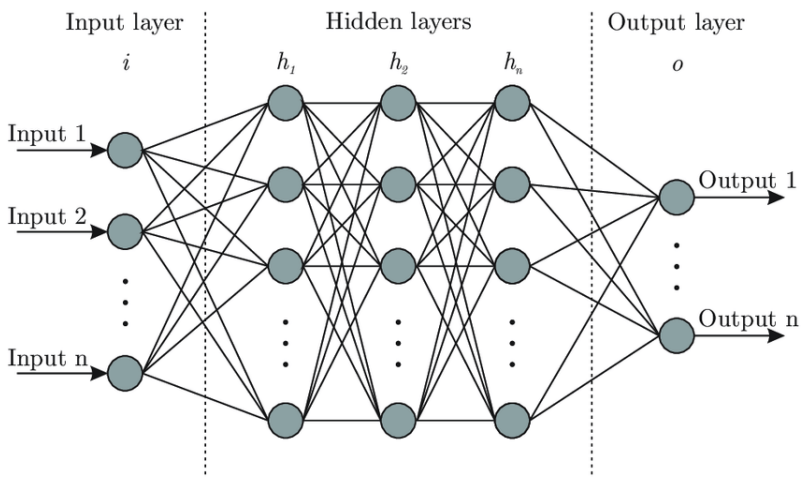
Công thức dự đoán nhãn:

$$y_{new} = \text{sgn} \left(\sum_{m \in S} \lambda_m y_m K(\mathbf{x}_m, \mathbf{x}) + b \right) \tag{2.8}$$

2.2 Convolutional Neural Network (CNN)

2.2.1 Mạng nơ-ron truyền thống (Fully Connected Network)

Trước khi đi sâu vào kiến trúc CNN, ta xem xét mô hình mạng nơ-ron truyền thống, nơi mỗi nơ-ron ở tầng này kết nối với toàn bộ nơ-ron ở tầng kế tiếp.



Hình 2.6: Mô hình mạng nơ-ron truyền thống với các tầng liên kết đầy đủ.

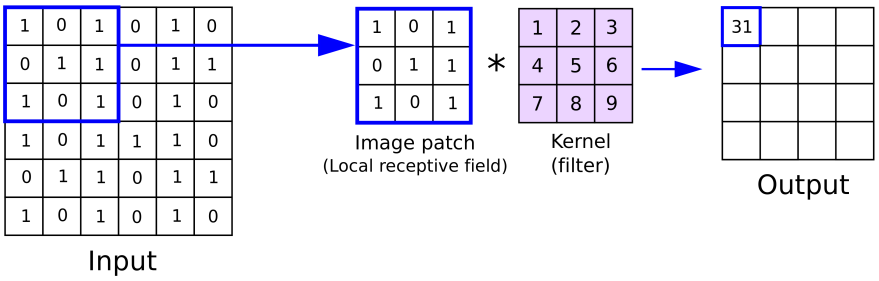
Tuy nhiên, đối với dữ liệu hình ảnh, mạng liên kết đầy đủ có hai hạn chế lớn: sự bùng nổ tham số do số lượng kết nối lớn, và mất thông tin không gian khi flatten ảnh thành vector.

2.2.2 Mạng nơ-ron tích chập (CNN)

CNN giải quyết các nhược điểm trên bằng cách sử dụng kết nối cục bộ và chia sẻ trọng số.

a, Kernel và Bias

CNN sử dụng bộ lọc (Kernel) trượt trên ảnh để thực hiện phép nhân chập.



Hình 2.7: Quá trình trượt Kernel trên ảnh đầu vào để trích xuất đặc trưng.

Công thức tích chập:

$$O_{i,j} = \sum_{m} \sum_{n} (I_{i+m,j+n} \cdot K_{m,n}) + b \tag{2.9}$$

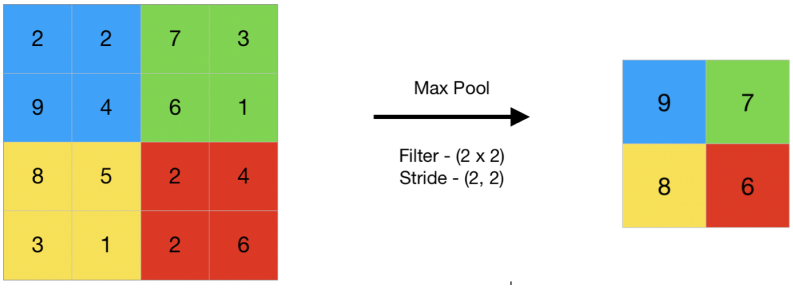
b, Hàm kích hoạt (Activation Function)

Hàm ReLU được sử dụng phổ biến:

$$f(x) = \max(0, x) \tag{2.10}$$

c, Lớp lấy mẫu cực đại (Max Pooling)

Max Pooling giảm kích thước dữ liệu bằng cách lấy giá trị lớn nhất trong vùng.



Hình 2.8: Minh họa cơ chế Max Pooling giúp giảm chiều dữ liệu.

d, Hàm mục tiêu và Phân loại (Loss Function)

Hàm Softmax chuyển logits thành xác suất:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}} \tag{2.11}$$

Hàm mất mát Cross-Entropy:

$$L = -\sum_{i=1}^{n} y_i \log(\hat{y}_i) \tag{2.12}$$

Mục tiêu là tối thiểu hóa L.

e, Lan truyền ngược (Backpropagation)

Quá trình lan truyền ngược sử dụng quy tắc chuỗi để tính đạo hàm và cập nhật các tham số trong kernel và bias.

Công thức cập nhật:

$$K_{new} = K_{old} - \eta \cdot \frac{\partial L}{\partial K} \tag{2.13}$$

$$b_{new} = b_{old} - \eta \cdot \frac{\partial L}{\partial b} \tag{2.14}$$

trong đó K là kernel và b là bias

CHƯƠNG 3. ÁP DỤNG VÀO BÀI TOÁN THỰC TẾ

3.1 Thực nghiệm với mô hình SVM

Trong phần này, chúng ta sẽ đi sâu vào quá trình triển khai mô hình SVM để phân loại bộ dữ liệu MNIST, từ khâu xử lý dữ liệu thô đến việc thiết lập cấu hình huấn luyện tối ưu.

3.1.1 Tiền xử lý dữ liệu (Preprocessing)

Dữ liệu hình ảnh từ bộ MNIST cần được đưa về định dạng phù hợp trước khi đưa vào mô hình SVM. Quy trình thực hiện gồm các bước chính sau:

- **Chuẩn hóa (Normalization):** Các giá trị pixel trong ảnh gốc nằm trong khoảng $[0, 255]$. Chúng ta thực hiện chuẩn hóa về đoạn $[0.0, 1.0]$ bằng công thức:

$$X_{norm} = \frac{X_{raw}}{255.0} \tag{3.1}$$

  Việc chuẩn hóa giúp các giá trị đặc trưng có cùng quy mô, hỗ trợ hàm Kernel RBF tính toán khoảng cách chính xác hơn và tăng tốc độ hội tụ của thuật toán.

- **Làm phẳng dữ liệu (Flattening):** SVM yêu cầu đầu vào là các vector đặc trưng một chiều. Do đó, mỗi ma trận ảnh kích thước 28×28 được trải phẳng thành một vector có độ dài 784:

$$X \in \mathbb{R}^{28\times 28} \xrightarrow{\text{Flatten}} X' \in \mathbb{R}^{784} \tag{3.2}$$

- **Quản lý nhãn:** Nhãn dữ liệu được giữ nguyên dưới dạng số nguyên từ 0 đến 9 để phục vụ cho chiến lược phân loại đa lớp.

3.1.2 SVM trong phân loại đa lớp

Về bản chất toán học, thuật toán SVM được thiết kế để giải quyết bài toán phân loại nhị phân (Binary Classification) — tức là chỉ phân biệt giữa hai lớp dữ liệu dựa trên một siêu phẳng tối ưu. Tuy nhiên, bộ dữ liệu MNIST có 10 lớp chữ số từ 0 đến 9. Để giải quyết vấn đề này, thư viện `sklearn.svm.SVC` áp dụng chiến lược **One-vs-One (OvO)**.

Cơ chế hoạt động của chiến lược One-vs-One: Trong chiến lược OvO, thay vì xây dựng một bộ phân loại duy nhất cho 10 lớp, hệ thống sẽ xây dựng các bộ phân loại nhị phân cho từng cặp lớp có thể có.

Số lượng bộ phân loại nhị phân cần thiết được tính theo công thức tổ hợp:

$$N_{classifiers} = \frac{k(k-1)}{2} \tag{3.3}$$

Trong đó k là số lượng lớp. Với bài toán MNIST ($k = 10$), số lượng bộ phân loại được tạo ra là:

$$N_{classifiers} = \frac{10(10-1)}{2} = 45 \text{ bộ phân loại nhị phân} \tag{3.4}$$

Cách ra quyết định của AI (Cơ chế bỏ phiếu - Voting): Khi một hình ảnh chữ số mới (vector X') được đưa vào hệ thống để dự đoán, quá trình ra quyết định diễn ra như sau:

1. **Dự đoán song song:** Hình ảnh này sẽ được đưa qua toàn bộ 45 bộ phân loại nhị phân đã huấn luyện.
2. **Tích lũy phiếu bầu:** Mỗi bộ phân loại sẽ đưa ra một "phiếu bầu" cho lớp mà nó cho là đúng nhất.
   - Ví dụ: Bộ phân loại "3 vs 8" nhận định ảnh là số 3, thì lớp "3" sẽ được cộng thêm 1 điểm.
3. **Quyết định cuối cùng:** Lớp nào nhận được nhiều phiếu bầu nhất từ tổng số 45 dự đoán sẽ được AI chọn làm nhãn dự đoán cuối cùng cho hình ảnh đó.

3.1.3 Triển khai (Implement)

a, Sơ đồ cấu trúc thư mục (Project Structure)

Dự án phân loại chữ số MNIST sử dụng SVM được tổ chức theo cấu trúc thư mục sau để đảm bảo tính mô đun và dễ bảo trì:

- **main.py:** File chính để chạy chương trình.
- **Data_preparation/:** Thư mục chứa các module xử lý dữ liệu (data_loader.py).
- **Models/:** Thư mục chứa module svm_model.py.
- **Results/:** Thư mục lưu trữ kết quả và mô hình (svm_model.joblib, Result SVM.txt).

b, Quá trình huấn luyện AI

Quá trình huấn luyện mô hình SVM được thực hiện theo các bước chi tiết sau:

1. **Tải và tiền xử lý dữ liệu:**

- Chuẩn hóa giá trị pixel từ khoảng [0, 255] về [0.0, 1.0].
- Làm phẳng mỗi ảnh 28×28 thành vector 784 chiều.
- Giữ nguyên nhãn dưới dạng số nguyên từ 0 đến 9.

2. **Huấn luyện mô hình:** Mô hình SVM được cấu hình với Kernel RBF, C=1.0, gamma='scale'.
3. **Lưu trữ mô hình:** Mô hình đã huấn luyện được lưu vào file Results/svm_model.joblib.

Các hàm huấn luyện, lưu trữ và đánh giá mô hình được triển khai trong module svm_model.py và được gọi thông qua hàm run_svm_project().

3.2 Thực nghiệm với mô hình CNN

Trong phần này, chúng ta sẽ khám phá việc áp dụng Mạng Nơ-ron Tích chập (CNN) để phân loại bộ dữ liệu MNIST. CNN tận dụng cấu trúc không gian của dữ liệu hình ảnh, mang lại hiệu suất vượt trội so với các phương pháp truyền thống như SVM.

3.2.1 Tiền xử lý dữ liệu (Preprocessing)

Dữ liệu hình ảnh MNIST cần được chuẩn bị phù hợp cho mạng CNN. Quy trình xử lý gồm các bước sau:

- **Chuẩn hóa (Normalization):** Các giá trị pixel từ $[0, 255]$ được chuẩn hóa về $[0.0, 1.0]$ bằng công thức:

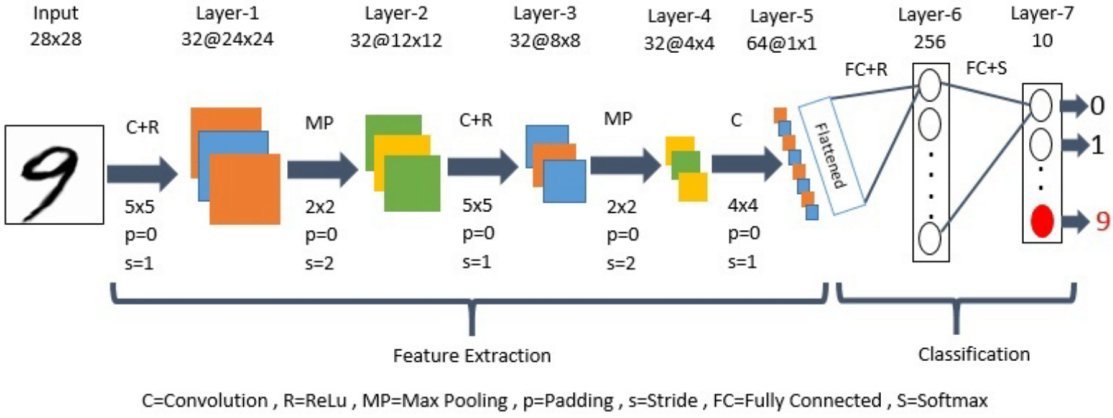
$$X_{norm} = \frac{X_{raw}}{255.0} \tag{3.5}$$

- **Reshape dữ liệu:** Dữ liệu được định hình lại thành tensor 3 chiều $(28 \times 28 \times 1)$ để phù hợp với đầu vào của CNN:

$$X \in \mathbb{R}^{784} \xrightarrow{\text{Reshape}} X' \in \mathbb{R}^{28\times28\times1} \tag{3.6}$$

- **Mã hóa nhãn (One-hot encoding):** Nhãn được chuyển đổi từ số nguyên sang vector one-hot.

3.2.2 Thiết kế kiến trúc mạng sequential



Hình 3.1: Minh họa kiến trúc mạng sequential trong mô hình CNN.

Bảng 3.1: Chi tiết cấu trúc các tầng trong mô hình CNN đề xuất

Tầng (Layer)	Tham số cấu hình	Chức năng
Conv2D_1	32 filters, size 3x3, ReLU	Trích xuất các đặc trưng cơ bản (cạnh, nét).
MaxPooling_1	Pool size 2x2	Giảm kích thước không gian, giữ đặc trưng mạnh nhất.
Conv2D_2	64 filters, size 3x3, ReLU	Trích xuất các đặc trưng phức tạp hơn.
MaxPooling_2	Pool size 2x2	Tiếp tục giảm chiều dữ liệu.
Conv2D_3	64 filters, size 3x3, ReLU	Trích xuất đặc trưng bậc cao.
Flatten	-	Chuyển đổi Tensor 3D thành Vector 1D.
Dense (Hidden)	64 units, ReLU	Kết hợp các đặc trưng để chuẩn bị phân loại.
Dense (Output)	10 units, Softmax	Trả về xác suất dự đoán cho 10 chữ số.

3.2.3 Triển khai (Implement)

a, Sơ đồ cấu trúc thư mục (Project Structure)

Dự án phân loại chữ số MNIST sử dụng CNN được tổ chức theo cấu trúc thư mục tương tự như mô hình SVM:

- **main.py:** File chính để chạy chương trình.
- **Data_preparation/:** Thư mục chứa module data_loader.py.
- **Models/:** Thư mục chứa module cnn_model.py.
- **Results/:** Thư mục lưu trữ kết quả và mô hình (mnist_cnn_model.h5, Result CNN.txt).

b, Quá trình huấn luyện AI

Quá trình huấn luyện mô hình CNN được thực hiện theo các bước chi tiết sau:

1. **Tải và tiền xử lý dữ liệu:**
   - Reshape dữ liệu thành (28, 28, 1).
   - Chuẩn hóa giá trị pixel về [0.0, 1.0].
   - Mã hóa nhãn thành dạng one-hot.
2. **Xây dựng kiến trúc mạng:** Mô hình bao gồm 3 lớp tích chập với MaxPooling, Flatten, và 2 lớp Dense. Sử dụng Optimizer Adam, Loss Categorical Crossentropy.
3. **Huấn luyện mô hình:** Với epochs: 5, Batch size: 64, Validation split: 10%.
4. **Lưu trữ mô hình:** Mô hình được lưu vào file Results/mnist_cnn_model.h5.

Các hàm xây dựng, huấn luyện, lưu trữ và đánh giá mô hình được triển khai trong module cnn_model.py và được gọi thông qua hàm run_cnn_project().

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM

Chương này trình bày các kết quả đạt được sau quá trình huấn luyện và đánh giá hệ thống trên bộ dữ liệu MNIST. Các số liệu được thu thập dựa trên môi trường thực thi thực tế để đưa ra cái nhìn khách quan về hiệu năng của các thuật toán.

4.1 Kết quả thực nghiệm trên mô hình SVM

Mô hình SVM với cấu hình Kernel RBF và chiến lược One-vs-One (OvO) đã được huấn luyện trên toàn bộ tập dữ liệu 60,000 mẫu và đánh giá trên 10,000 mẫu kiểm tra.

4.1.1 Hiệu năng tổng quát

Các thông số về thời gian thực thi và độ chính xác tổng quát của mô hình được tóm tắt trong Bảng 4.1.

Bảng 4.1: Hiệu năng tổng quát của mô hình SVM

Thông số	**Giá trị thực nghiệm**
Thời gian huấn luyện (Training Time)	194.61 giây
Thời gian dự đoán trung bình	$\approx$ 0.02 s/ảnh
Độ chính xác cuối cùng (Final Accuracy)	**97.9200%**

Mô hình SVM với Kernel RBF đạt độ chính xác 97.92% sau khoảng 194.61 giây huấn luyện, khẳng định hiệu quả của phương pháp trong không gian đặc trưng cao chiều.

4.1.2 Phân tích ma trận nhầm lẫn (Confusion Matrix)

Ma trận nhầm lẫn giúp quan sát chi tiết khả năng phân loại của mô hình trên từng lớp chữ số, đồng thời xác định các cặp chữ số dễ gây nhầm lẫn nhất.

Bảng 4.2: Ma trận nhầm lẫn của mô hình SVM trên 10,000 mẫu kiểm tra

Thực tế \ Dự đoán	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
0	973	0	1	0	0	2	1	1	2	0
1	0	1126	3	1	0	1	1	1	2	0
2	6	1	1006	2	1	0	2	7	6	1
3	0	0	2	995	0	2	0	5	5	1
4	0	0	5	0	961	0	3	0	2	11
5	2	0	0	9	0	871	4	1	4	1
6	6	2	0	0	2	3	944	0	1	0
7	0	6	11	1	1	0	0	996	2	11
8	3	0	2	6	3	2	2	3	950	3
9	3	4	1	7	10	2	1	7	4	970

Dựa trên Bảng 4.2, SVM gặp khó khăn nhất với cặp (4, 9) (tổng cộng 21 trường hợp nhầm lẫn) và cặp (2, 7) (11 trường hợp nhầm 7 thành 2). Sai số chủ yếu do sự tương đồng về nét vẽ.

4.1.3 Chỉ số chi tiết (Classification Report)

Các chỉ số Precision, Recall và F1-score giúp đánh giá sự cân bằng của mô hình trên toàn bộ 10 lớp chữ số.

Bảng 4.3: Báo cáo kết quả chi tiết theo từng lớp (SVM)

Lớp (Digit)	**Precision**	**Recall**	**F1-score**	**Số lượng mẫu**
0	0.98	0.99	0.99	980
1	0.99	0.99	0.99	1135
2	0.98	0.97	0.98	1032
3	0.97	0.99	0.98	1010
4	0.98	0.98	0.98	982
5	0.99	0.98	0.98	892
6	0.99	0.99	0.99	958
7	0.98	0.97	0.97	1028
8	0.97	0.98	0.97	974
9	0.97	0.96	0.97	1009
Trung bình	**0.98**	**0.98**	**0.98**	**10,000**

Chỉ số F1-score duy trì ở mức cao (0.97 đến 0.99) cho tất cả các lớp, khẳng định sự ổn định và cân bằng của chiến lược One-vs-One.

4.2 Kết quả thực nghiệm trên mô hình CNN

Mô hình CNN được huấn luyện thông qua 5 chu kỳ (epochs) với dữ liệu tensor 4D. Kết quả cho thấy tốc độ hội tụ nhanh và khả năng tổng quát hóa dữ liệu xuất sắc.

4.2.1 Hiệu năng tổng quát và Quá trình hội tụ

Hiệu năng của mạng CNN được ghi lại chi tiết qua từng epoch, cho thấy sự sụt giảm ổn định của hàm mất mát (loss) và sự tăng trưởng của độ chính xác (accuracy).

Bảng 4.4: Tóm tắt hiệu năng huấn luyện và kiểm tra của CNN

Thông số	**Giá trị thực nghiệm**
Số lượng Epochs	5
Thời gian huấn luyện tổng cộng	**47.16 giây**
Thời gian trung bình mỗi bước	11 ms/step
Độ chính xác trên tập kiểm tra (Accuracy)	**99.0500%**

So với SVM, mô hình CNN đạt độ chính xác cao hơn (99.05%) và thời gian huấn luyện ngắn hơn đáng kể (47.16 giây), minh chứng cho ưu thế của việc trích xuất đặc trưng không gian.

4.2.2 Phân tích ma trận nhầm lẫn (Confusion Matrix)

Ma trận nhầm lẫn của CNN cho thấy các điểm dữ liệu bị phân loại sai đã giảm đi đáng kể so với SVM.

Bảng 4.5: Ma trận nhầm lẫn của mô hình CNN trên 10,000 mẫu kiểm tra

Thực tế \ Dự đoán	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
0	977	0	1	0	0	0	0	1	1	0
1	0	1130	1	1	1	0	1	0	0	1
2	0	0	1029	1	1	0	0	1	0	0
3	0	0	4	1001	0	3	0	0	1	1
4	0	0	0	0	968	0	3	1	3	7
5	1	0	1	5	0	882	1	0	2	0
6	4	2	0	0	2	4	944	0	2	0
7	0	2	13	1	0	0	0	1007	1	5
8	2	0	1	2	0	0	0	0	968	1
9	0	0	0	0	3	3	0	1	3	999

CNN giảm đáng kể các lỗi nhầm lẫn. Cặp (7, 2) vẫn là lỗi phổ biến nhất (13 trường hợp nhầm 7 thành 2). Quan trọng hơn, lỗi nhầm lẫn cặp (4, 9) đã giảm xuống chỉ còn 7 trường hợp (so với 21 ở SVM).

4.2.3 Chỉ số chi tiết (Classification Report)

Bảng chỉ số chi tiết dưới đây minh chứng cho sự ổn định của mô hình trên toàn bộ 10 chữ số.

Bảng 4.6: Báo cáo kết quả chi tiết theo từng lớp (CNN)

Lớp (Digit)	Precision	Recall	F1-score	Số lượng mẫu
0	0.99	1.00	0.99	980
1	1.00	1.00	1.00	1135
2	0.98	1.00	0.99	1032
3	0.99	0.99	0.99	1010
4	0.99	0.99	0.99	982
5	0.99	0.99	0.99	892
6	0.99	0.99	0.99	958
7	1.00	0.98	0.99	1028
8	0.99	0.99	0.99	974
9	0.99	0.99	0.99	1009
Trung bình	**0.99**	**0.99**	**0.99**	**10,000**

F1-score đạt mức 0.99 cho hầu hết các lớp và đạt mức tuyệt đối 1.00 cho lớp số 1, cho thấy khả năng phân loại vượt trội của mô hình CNN.

CHƯƠNG 5. DEMO

Chương này trình bày chi tiết quá trình thực nghiệm, bao gồm việc huấn luyện các mô hình SVM và CNN và kiểm tra hiệu suất của chúng trên tập dữ liệu MNIST.

5.1 Huấn luyện mô hình

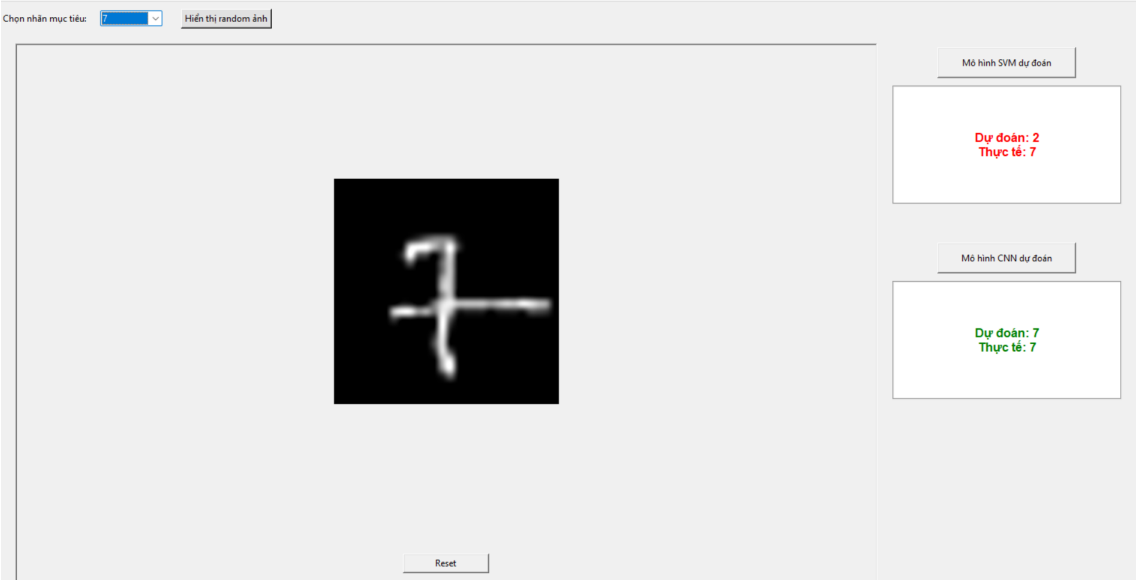
Quá trình huấn luyện hai mô hình SVM và CNN được thực thi thông qua các hàm `run_svm_project()` và `run_cnn_project()` trong file `main.py`. Kết quả độ chính xác và ma trận nhầm lẫn chi tiết được hiển thị trực tiếp trên terminal và đã được tổng hợp trong Chương 4.

5.2 Kiểm tra mô hình

Để đánh giá hiệu suất của hai mô hình, em đã thiết kế một giao diện ứng dụng cho phép lấy ngẫu nhiên các ảnh chữ số dựa trên nhãn được chọn trước để tiến hành kiểm tra.

Hình 5.1: SVM và CNN đều nhận diện đúng số 3

Hình 5.2 minh họa trường hợp mô hình SVM nhận diện sai chữ số 7 thành số 2, nhưng mô hình CNN vẫn nhận diện chính xác, minh chứng cho khả năng học đặc trưng hình học mạnh mẽ hơn của CNN.



Hình 5.2: SVM nhận diện sai số 7 thành số 2, CNN nhận diện đúng

CHƯƠNG 6. KẾT LUẬN

Sau quá trình nghiên cứu, triển khai và thực nghiệm bài toán nhận diện chữ số viết tay trên bộ dữ liệu MNIST, báo cáo đã hoàn thành các mục tiêu đề ra với những kết quả cụ thể như sau:

- **Về mặt kỹ thuật:** Đã làm chủ và triển khai thành công hai phương pháp tiếp cận tiêu biểu: Học máy truyền thống (SVM với Kernel RBF) và Học sâu (CNN). Hệ thống đã được tổ chức theo mô hình mô-đun hóa chuyên nghiệp, từ khâu tiền xử lý dữ liệu đến huấn luyện và lưu trữ mô hình.
- **Về mặt thực nghiệm:**
  - Mô hình CNN cho thấy ưu thế vượt trội với độ chính xác đạt **99.05%** và thời gian huấn luyện tối ưu (47.16 giây).
  - Mô hình SVM đạt độ chính xác ổn định **97.92%**, khẳng định hiệu quả của chiến lược One-vs-One trong các bài toán phân loại đa lớp.
- **Về mặt ứng dụng:** Xây dựng thành công ứng dụng minh họa (Demo) với giao diện Tkinter, cho phép người dùng tương tác trực quan và kiểm chứng kết quả dự đoán của các mô hình trong thời gian thực.